

Systèmes d'exploitation II

Chapitre 1 : Ordonnancement des processus

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

- 1 Introduction - rappels et notions élémentaires
- 2 Différents types d'ordonnanceurs
- 3 Algorithmes de scheduling
- 4 Étude de cas : SOLARIS & Linux

Objectifs :

- Comprendre les notions fondamentales de l'ordonnancement des systèmes d'exploitation
- Identifier les différents types d'ordonnanceurs et leurs rôles
- Expliquer et comparer les principaux algorithmes de scheduling
- Analyser le comportement des ordonnanceurs à travers des études de cas réels : Solaris et Linux

Introduction

★ Même avec un seul processeur, l'utilisateur a l'impression que tous les programmes s'exécutent en même temps. Cette illusion repose sur :

- une **commutation** rapide entre processus,
- un partage temporel finement contrôlé du processeur,
- l'exploitation des périodes d'attente d'E/S.

→ Comment partager **une ressource unique** (le CPU) entre plusieurs processus de manière **efficace, équitable et réactive** ?

L'**ordonnanceur (scheduler)** est la composante de l'OS chargée de décider **quel processus** obtient le CPU. Il s'appuie sur :

- des algorithmes,
- des politiques de partage,
- des structures de données noyau.

Commutation de contexte (1/2)

Le **contexte d'un processus** regroupe l'ensemble des informations nécessaires pour reprendre son exécution exactement là où elle a été interrompue. Il est stocké dans le PCB (Process Control Block) :

- **Contexte utilisateur** : zones texte, données et pile utilisateur
- **Contexte matériel** : l'ensemble des registres du processeur
- **Contexte système** : les structures de données du système

La **commutation de contexte** consiste à :

- sauvegarder le contexte du processus courant,
 - restaurer le contexte d'un autre processus.
-
- **inévitable** dans un système multiprogrammé : blocage sur une E/S, une interruption, une exception, ...
 - **nécessaire** pour exploiter le recouvrement CPU / E/S,
 - **indispensable** pour la réactivité des systèmes interactifs,
 - **essentielle** pour assurer le partage et l'équité du CPU dans un système multiprogrammé.

Commutation de contexte (2/2)

Synchrone

- **involontaire**: le processeur est alloué à un autre processus
 - ▶ interruption d'horloge
- **volontaire**: un processus se bloque (appel système sleep())

Asynchrone

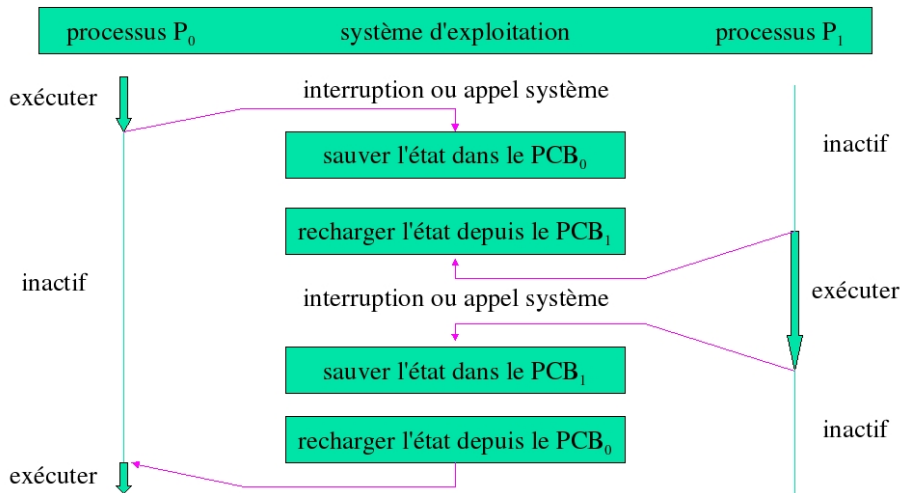
- cas d'une interruption matérielle
- la commutation est généralement prise en charge par le matériel

Le **dispatcher** est le module qui transfère effectivement le CPU au processus choisi par l'ordonnanceur :

- commutation de contexte (en mode noyau souvent suite à une interruption)
- commutation vers le mode utilisateur
- branchement vers l'instruction de reprise du processus élu.

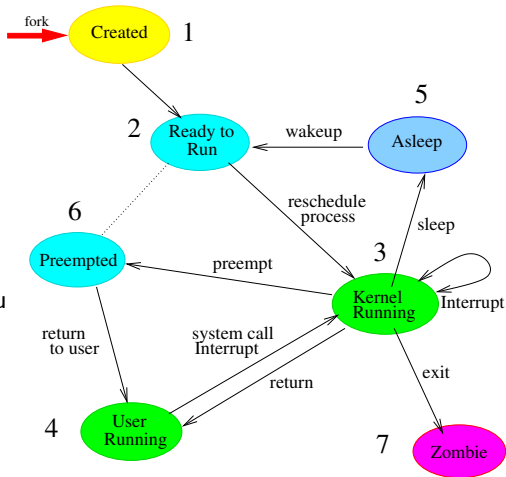
→ Minimiser la latence de dispatching.

Commutation de contexte illustrée



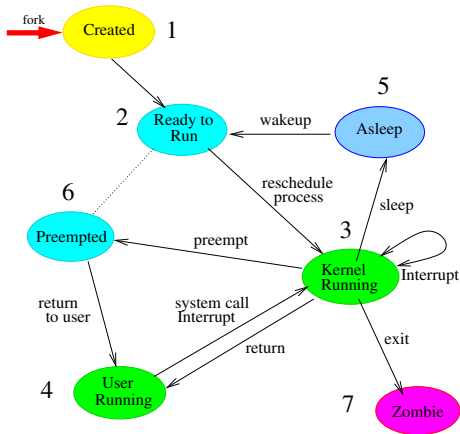
États d'un processus

1. **Created**: en cours de création
2. **Ready** : le processus est prêt en attente de l'attribution du processeur
3. **Kernel Running** : le processus est exécuté en mode noyau
4. **User Running** : le processus est exécuté en mode utilisateur
5. **Asleep/blocked** : le processus en attente d'un événement (I/O, synchronisation, ressource)
6. **Preempted (Réquisitionné)** : le noyau lui a réquisitionné le processeur lors du passage du mode noyau au mode utilisateur.
7. **Zombie (Terminé)** : le processus est terminé mais conserve des informations résiduelles en attendant leur récupération par le processus parent.



États d'un processus - Exemples de transitions

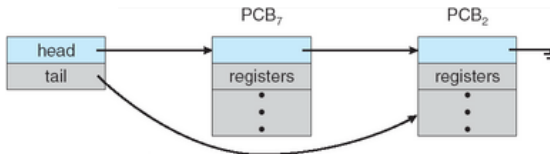
- 1 → 2 : processus admis dans le système et devient prêt à être exécuté
- 2 → 3 : l'ordonnanceur a choisi le processus et s'exécute en mode noyau pour terminer le fork()
- 3 → 4 : le processus termine l'appel système et passe en mode utilisateur
- 4 → 3 : interruption d'horloge et retour au mode noyau
- 3 → 6 : interruption traitée et l'ordonnanceur décide de passer le processeur à un autre processus
- 6 → 4 : l'ordonnanceur a choisi ce processus



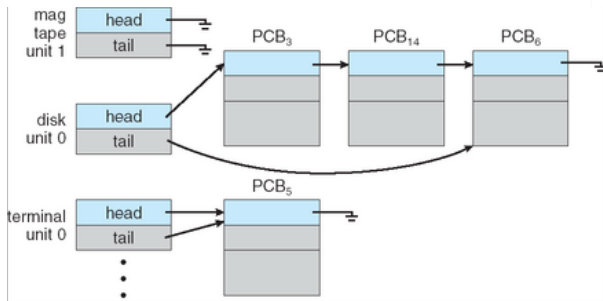
★ Il s'agit d'une description abstraite - la mise en œuvre concrète dépend de l'OS.
ex. . Les états 2 et 6 sont séparés pour montrer qu'un processus en mode noyau ne peut être préempté que lorsqu'il retourne au mode utilisateur.

Files d'attente des processus

- ★ [Job queue] Ensemble des processus du système
- ★ [Ready queue] Ensemble des processus prêts à être exécutés par le processeur et résidant en mémoire principale.
- ★ [Device queues] Ensembles de processus en attente de la fin d'une opération d'entrée/sortie ou de la disponibilité d'une ressource.

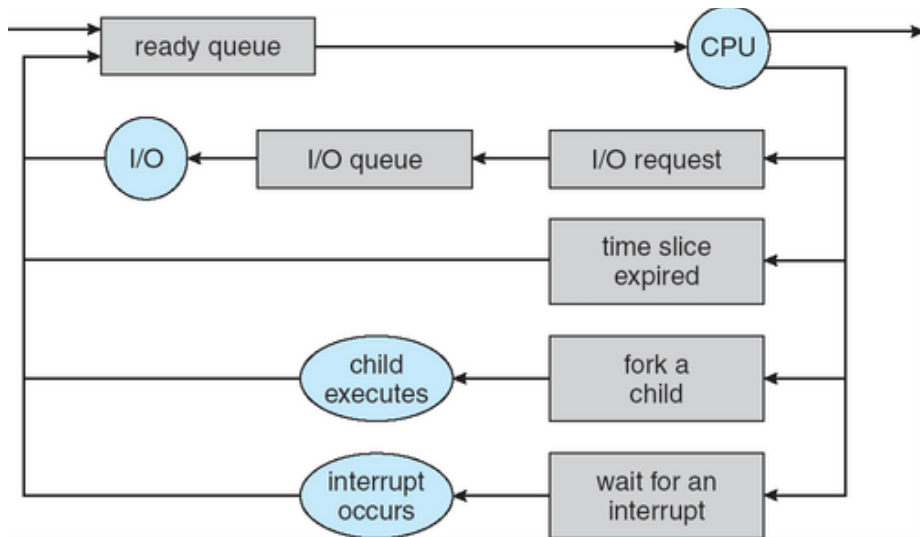


Ready queue



Device queues

Files d'attente des processus - évolution



Deux ordonnanceurs ...

★ Ordonnanceur à long terme

Ordonnanceur des travaux
(Long-term / Job Scheduler)

- choisit quels processus de la [job queue] → [ready queue]
- contrôle le degré de multiprogrammation
- invoqué peu fréquemment (secondes, minutes)
- peut être relativement lent

★ Ordonnanceur à court terme

Ordonnanceur de l'UC
(Short-term / CPU scheduler)

- choisit un processus de la [ready queue] à exécuter
- attribue le processeur (CPU)
- invoqué très fréquemment (millisecondes)
- doit être très rapide

Ordonnancement : quand ?

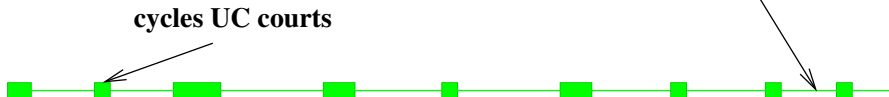
- ❶ “en cours d’exécution” → “en attente” (attente d’un évènement)
 - ❷ “en cours d’exécution” → “prêt” (suite à une interruption)
 - ❸ “en attente” → “prêt” (terminaison d’une E/S par exemple)
 - ❹ Terminaison d’un processus
- 1 et 4 correspondent à un ordonnancement **sans réquisition** ou **non-preemptive scheduling**
 - 2 et 3 correspondent à un ordonnancement **avec réquisition** ou **preemptive scheduling**
⇒ problèmes d’exclusion mutuelle à résoudre

Comportement des processus (1/2)

Processus tributaire de l'UC



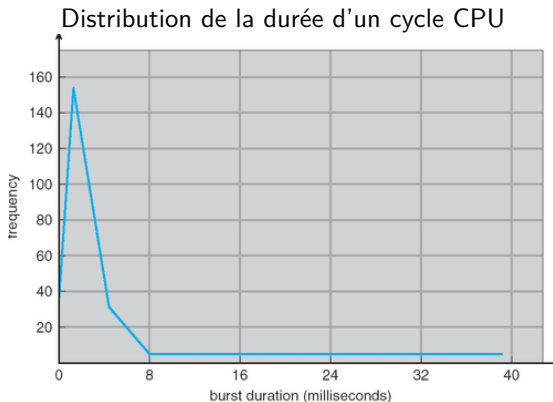
Processus tributaire d'E/S



cycles d'E/S

- Avoir une idée sur le type de processus sur un système donné permet de choisir l'algorithme d'ordonnancement le plus approprié

Comportement des processus (2/2)



- Les cycles CPU ont tendance à être plus fréquents et plus courts dans un système interactif.

Types de systèmes et stratégies d'ordonnancement

- Systèmes de traitement par lots (batch)
 - ▶ un algorithme **sans réquisition** (non préemptif) est suffisant
 - ▶ les commutations de contextes sont réduites → performances améliorées
- Systèmes interactifs
 - ▶ un algorithme **avec réquisition** est nécessaire
 - ▶ garantit une **bonne réactivité** pour l'utilisateur
- Systèmes temps réel
 - ▶ décisions sont guidées par les **contraintes temporelles**
 - ▶ la réquisition est parfois inutile

- **Utilisation UC**: taux d'utilisation à maintenir au maximum possible
- **Capacité de traitement**: Processus terminés par unité de temps
- **Délai de rotation**: l'intervalle entre le moment de soumission d'un travail jusqu'au moment de sa terminaison
- **Temps d'attente**: la somme du temps passé à attendre dans la file d'attente des processus prêts
- **Temps de réponse**: temps écoulé entre la soumission d'une requête et l'arrivée de la première réponse
- **Équité**: partage équitable du CPU

Objectifs de l'ordonnancement

Tous les systèmes

- Optimisation de l'utilisation du CPU :
 - ▶ éviter l'inactivité du processeur
 - ▶ exploiter le recouvrement CPU / E/S
- Partage du CPU entre plusieurs processus
- Appliquer la politique d'ordonnancement
- Équité entre les processus
- Maximiser la capacité de traitement

Systèmes de traitement par lots

- Minimiser le délai de rotation
- Maximiser l'utilisation du processeur

Systèmes temps réel

- Respecter les délais (objectif principal)
- Prévisibilité
- Qualité de service (**ex.** multimédia)

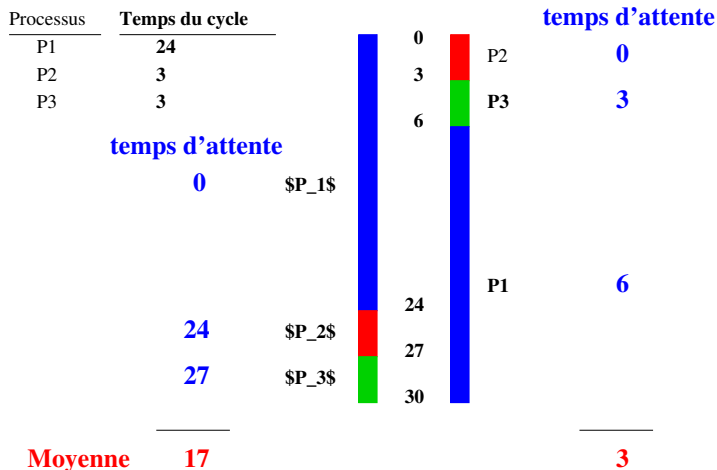
Systèmes interactifs

- Assurer une **bonne réactivité**
- Minimiser le temps de réponse
- Proportionnalité (répondre aux attentes des utilisateurs)

Algorithmes de scheduling

Premier arrivé, premier servi

- simple à réaliser à l'aide d'une file d'attente FIFO des PCBs
- l'UC est alloué au premier processus de la file d'attente
- pas de réquisition



Algorithmes de scheduling

Premier arrivé, premier servi

- le temps moyen d'attente peut varier beaucoup si le temps des cycles UC varie trop
- pénalise les processus d'E/S par rapport aux processus de traitement
- pas adapté aux systèmes interactifs
- et si on servait en premier le processus avec le prochain cycle le plus court ?

Algorithmes de scheduling

Le programme le plus court d'abord

(Shortest-Job-First)

- Associer à chaque processus la longueur de son prochain cycle UC
- L'UC est assigné au processus qui a le prochain cycle UC le plus court
- SJF est optimal : minimise le temps d'attente moyen. À condition que tous les travaux (cycles UC) sont disponibles en même temps
- **Problème** : pénalise les gros travaux (les processus tributaires d'UC)
- Comment peut-on connaître le temps des cycles prochains ?

Algorithmes de scheduling

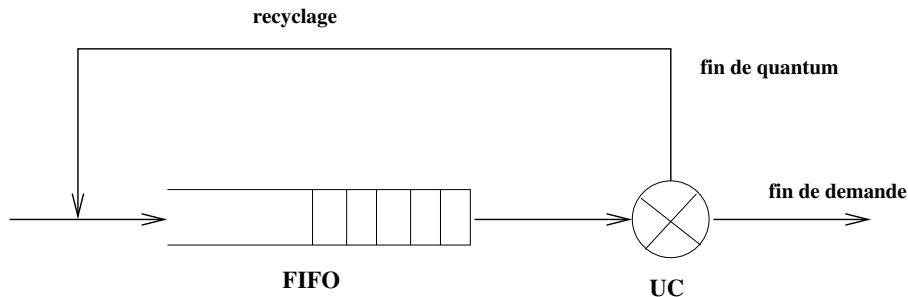
Scheduling avec des priorités

- associer à chaque processus une priorité
 - l'UC est allouée à celui qui a la plus haute priorité
 - peut être avec ou sans réquisition
-
- Problème : la **famine**. Une solution : **le vieillissement**(AGING)
 - **Cas particulier = FCFS** : même priorité pour tous
 - **Cas particulier = SJF** : la priorité d'un processus est inversement proportionnelle au temps de son prochain cycle.

Algorithmes de scheduling

Le tourniquet (round-robin)

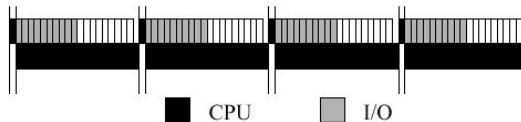
- L'UC est allouée par tranches de temps appelés **quantum**
- Si au bout du quantum, un processus n'a pas terminé, il est interrompu et inséré à la fin de la file des processus prêts
- Implémentation : une file FIFO avec recyclage des demandes



Algorithmes de scheduling

Le tourniquet : choix du quantum

Le quantum est grand

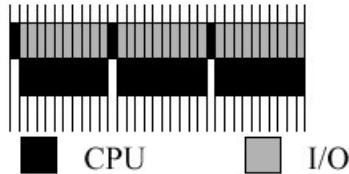


- processus 1 pénalisé (tourne à 11/21)
- mauvaise utilisation du périphérique d'E/S (10/21 au lieu de 10/11)
- en général : un quantum long augmente le débit global du système
- Remarque: lorsque $q = \infty$, RR devient FCFS

Algorithmes de scheduling

Le tourniquet : choix du quantum

Le quantum est petit



- le processus 2 est interrompu 9 fois pour rien
- **en général**: un quantum très court favorise les traitements interactifs mais trop court diminue les performances à cause de l'overhead dû aux changements de contexte
- RR suppose que tous les processus sont équivalents chacun reçoit une part égale de CPU. Ceci produit des mauvais résultats

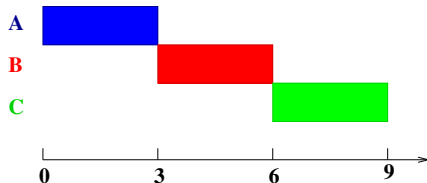
Latence vs Débit.

- un petit quantum améliore la latence (réactivité)
- un grand quantum améliore le débit (moins de commutations de contexte).

Algorithmes de scheduling

Le tourniquet : délai de rotation

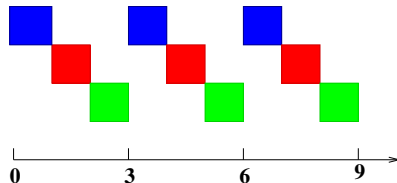
FIFO



Délai de rotation moyen

$$(3+6+9)/3=6$$

RR



Délai de rotation moyen

$$(7+8+9)/3=8$$

- Le délai de rotation moyen est long dans RR
- **Solution** : introduire des priorités
 - ▶ choisir le processus avec la priorité la plus élevée
 - ▶ faire du RR pour les processus à priorité égale

Algorithmes de scheduling

Multilevel Scheduling

- Les processus sont divisés en classes de processus
 - ▶ **Critères:** temps de réponse, taille mémoire, priorité ou type de processus ...
- Chaque classe de processus est assignée à une file d'attente séparée
- Chaque classe est schedulée avec un algorithme différent
- On peut associer une priorité différente à chaque file
- Un processus est toujours associé à la même file d'attente

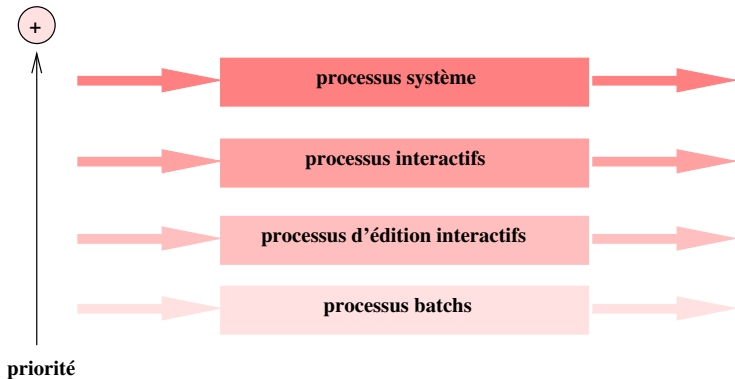
Exemple

- 2 classes :
 - ▶ RR pour les processus interactifs
 - ▶ FCFS pour les processus batch
- Un scheduling entre les files d'attente est nécessaire :
 - ▶ avec réquisition et priorités fixes. **ex.** : la 1ère classe est plus prioritaire
 - ▶ associer un quantum différent par file.
ex. : 80% du temps CPU à la 1ère classe et 20% pour la 2ème

Algorithmes de scheduling

Feedback Multilevel Scheduling

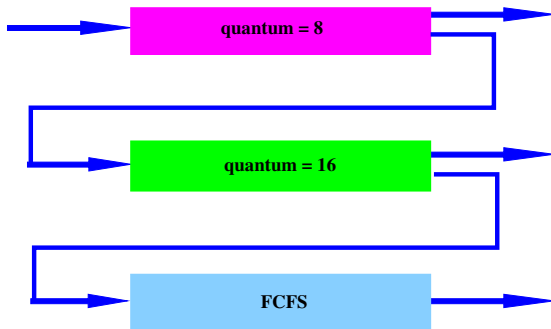
Exemple 2



Algorithmes de scheduling

Feedback Multilevel Scheduling

- Ici, les processus peuvent changer de files, ce qui permet de séparer les processus ayant des caractéristiques différentes en termes de cycles UC
- Si un processus a des cycles UC longs, le déplacer dans une file d'attente moins prioritaire $\Rightarrow$ les processus interactifs finissent par avoir la priorité la plus élevée
- Si un processus attend longtemps, il est déplacé dans une file plus prioritaire

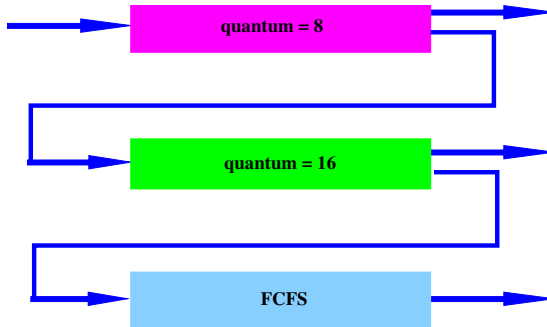


Algorithmes de scheduling

Feedback Multilevel Scheduling

est défini par

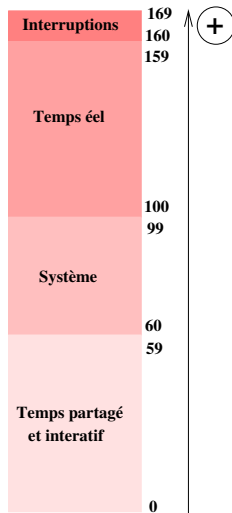
- le nombre de files d'attente
- l'algorithme de scheduling pour chaque file
- la méthode utilisée pour déterminer le moment de changer la priorité d'un processus



Ordonnancement SOLARIS

3 classes d'ordonnancement

- **Timesharing and interactive (TS & IA)** : RR avec priorité (plus de priorité aux processus les plus interactifs)
- **System(SYS)** : FIFO avec préemption et priorités fixes
- **Realtime (RT)** : RR avec priorité où un processus (RT) a une priorité fixe durant sa vie



Ordonnancement SOLARIS

Dispatch table : processus interactifs

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

- **time quantum**: la longueur par défaut du quantum assigné au processus
- **time quantum expired**: la nouvelle priorité pour un processus qui utilise la totalité de son quantum
- **return from sleep** : la nouvelle priorité pour un processus qui se bloque avant d'utiliser la totalité de son quantum

Linux : multiples ordonnanceurs

★ **Ordre de priorité (qui s'exécute en premier) :**

- **Early Deadline First [EDF]** - **SCHED_DEADLINE** : ordonnancement basé sur des échéances explicites, offrant des garanties temporelles.
- **Real Time [RT]** - priorités strictes 1 à 99
 - ▶ **SCHED_FIFO** une tâche s'exécute tant qu'aucune tâche de priorité supérieure n'est prête.
 - ▶ **SCHED_RR** partage équitable du processeur entre tâches de même priorité (quantum).
- **Completely Fair Scheduler [CFS]**
Earliest Eligible Virtual Deadline First [EEVDF] - équité
 - ▶ **SCHED_NORMAL** tâches interactives et conventionnelles
 - ▶ **SCHED_BATCH** tâches non interactives orientées débit
 - ▶ **SCHED_IDLE** exécuté uniquement en l'absence de tâches des autres classes

➔ La commande **chrt** permet de classer un processus dans l'une des 3 classes.

★ **Placement CPU (où la tâche s'exécute) :**

- **Capacity Aware Scheduler [CAS]** : placement des tâches en fonction de leur charge et de la capacité effective des cœurs.
- **Energy Aware Scheduler [EAS]** : Placement optimisé pour minimiser la consommation énergétique à performance équivalente.

Contrôle de l'ordonnancement

★ La notion de `nice` (tâches CFS / EEVDF)

- La valeur `nice` exprime la "gentillesse" d'un processus.
- Plage : `-20` (priorité élevée) à `+19` (priorité faible).
- `nice` n'est pas une priorité stricte : elle est traduite en *poids* utilisé par CFS / EEVDF.

★ Commandes utilisateur

- `nice` : lance un programme avec une valeur `nice`.
- `renice` : modifie la valeur `nice` d'un processus existant.
- `chrt` : configure la `classe d'ordonnancement` (CFS, RT, DEADLINE) et les paramètres associés.

★ Interface de programmation

- `setpriority()` : modifie la valeur `nice`.
- `sched_setattr()` : interface moderne (RT, DEADLINE, EEVDF)
- ...

Linux : ordonnancement des processus

CFS - Completely Fair Scheduler

- Depuis Linux 2.6.23, l'ordonnanceur des processus SCHED_NORMAL est le **CFS**.
- Objectif : **fairness proportionnelle** à la part de CPU attendue, tout en gardant une bonne **réactivité**.
- Idée centrale : chaque tâche accumule un **temps virtuel** (vruntime) qui représente le CPU "consommé" de manière normalisée.
- La tâche choisie est celle qui a le **plus petit vruntime** : celle qui a le "moins tourné".

Intuition

Le CPU va en priorité à celui qui est le plus "en retard" par rapport à un partage équitable.

Linux : CFS

vruntime et partage proportionnel

- Le vruntime augmente pendant l'exécution.
- Son incrément dépend du **poids** de la tâche (lié à nice) :
 - ▶ une tâche **plus prioritaire** (nice plus faible) a un poids plus grand, donc son vruntime *augmente plus lentement* ⇒ elle obtient *plus* de CPU.
 - ▶ une tâche **moins prioritaire** (nice plus élevé) voit son vruntime augmenter plus vite ⇒ moins de CPU.
- En régime " stable ", le CPU est partagé **au prorata des poids**.

Lecture rapide

nice ne donne pas un "quantum fixe", mais une **part** de CPU via le vruntime.

Linux : CFS

Quelle structure pour choisir le prochain ?

- Les tâches `SCHED_NORMAL` prêtes sont organisées par `vruntime` dans une structure ordonnée.
- Implantation (conceptuellement) : `arbre équilibré` (*red-black tree*).
 - ▶ insertion / retrait : $O(\log n)$
 - ▶ choix du prochain : le minimum (gauche de l'arbre)
- La réquisition intervient notamment :
 - ▶ si une tâche " en retard " (plus petit `vruntime`) devient prête,
 - ▶ si la tâche courante a consommé sa part sur la fenêtre de temps.